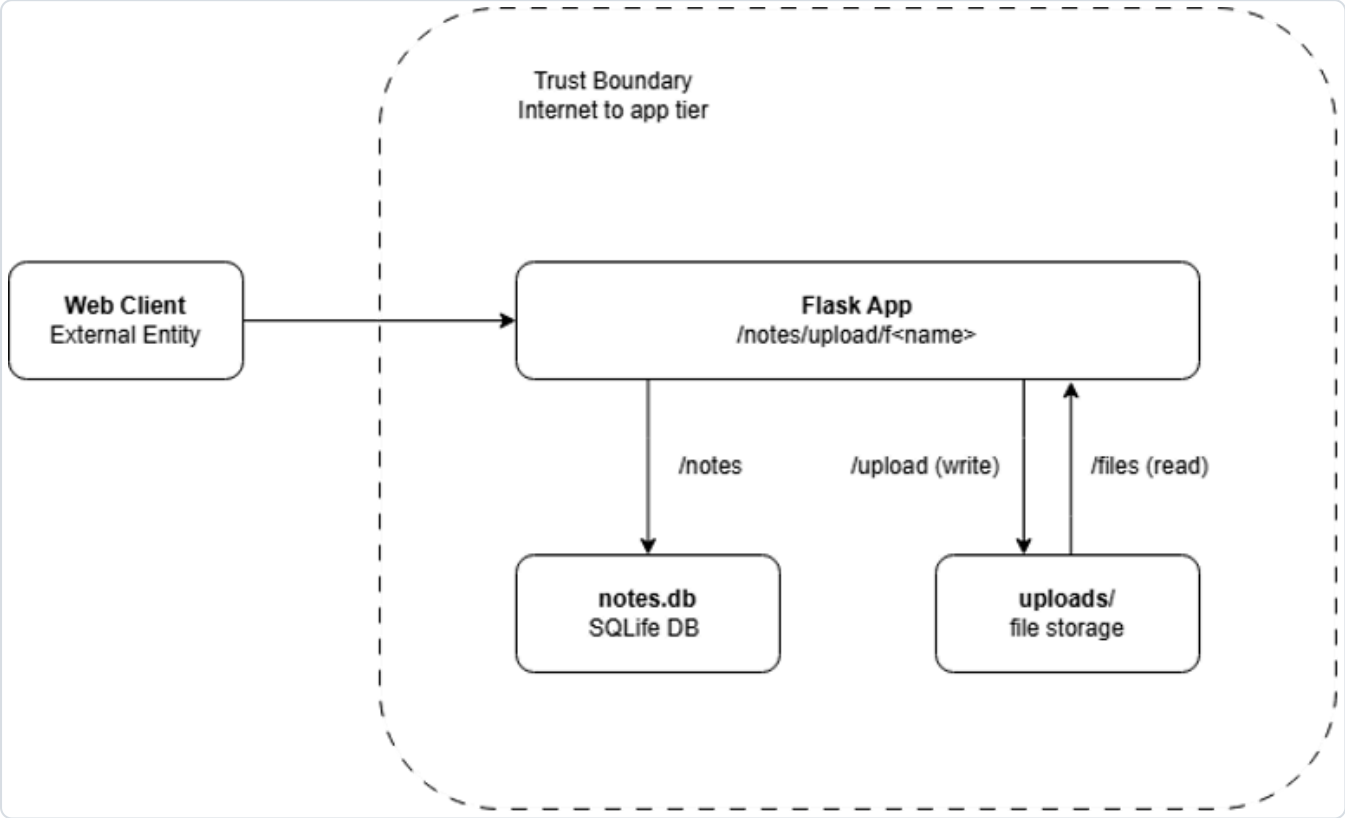
Threat Model —

1. Data-flow diagram

(Insert your DFD image. Mark trust boundaries with dashed lines.)



2. Elements & trust boundaries

Element	Type (process/store/entity/flow)	Trust boundary crossed?
Web client	external entity	yes (Internet → app)
Flask app	process	yes (Internet -> Flask app)
SQLite DB (`notes.db`)	data store	No (Internet -> Flask app ->notes.db)
`uploads/` store	data store	No (Internet -> Flask app -> uploads)

3. STRIDE analysis

Element	S	T	R	I	D	E
/notes	S	T	R	I	D	-
/upload	-	T	R	I	D	-
/files/	-	-	-	I	-	-

4. Top 5 risks (likelihood × impact) + mitigation

1. Tampering : arbitrary file write via /upload (High x Critical) + Use werkzeug.utils.secure_filename() with the filename + allow-list extension (.txt, .png only) + verify that the resolved path is still in uploads/ (canonicalize the path and then compare).
2. Information Disclosure : GET /notes return every note to everyone (High x High) + Add authentication (session/token) and then filter the query to WHERE owner = current_user.
3. Spoofing : POST /notes are accessing owner from client directly (High x Medium) + Get the owner from the authenticated session instead of getting it from the client JSON.
4. Denial of Service : these aren’t size/rate limit on /upload (Medium x Medium) + Set MAX_CONTENT_LENGTH in Flask config + add rate limiting (e.g., Flask-Limiter)
5. Repudiation : These are not logging operate (N/A x Medium) + Add request logging (method, path, IP, timestamp, user) through Flask logging middleware.